# MODUL PRAKTIKUM 8

## Array Satu Dimensi dan Sequential Search

Praktikum Dasar Pemrograman

Program Studi S1 Informatika

Semester 1

*Lingkungan Praktikum: Visual Studio Code + MSYS2 (GCC/GDB)*

Disusun oleh:

**Dr. Desta Sandya Prasvita, S.Komp., M.Kom.**

Modul ini melanjutkan materi fungsi dan pemrograman modular pada Modul 7. Fokus praktikum adalah menyimpan sekumpulan data bertipe sama menggunakan array satu dimensi, mengakses dan mengolah setiap elemen melalui perulangan, serta mencari data menggunakan algoritma sequential search.

Materi ini mendukung bahan kajian array satu dimensi, pengolahan kumpulan data, dan algoritma pencarian berurutan dalam RPS Algoritma dan Pemrograman.

# I. Pendahuluan Modul

## 1. Identitas Modul

| Komponen | Keterangan |
|---|---|
| Mata Kuliah | Praktikum Dasar Pemrograman |
| Program Studi | S1 Informatika |
| Semester | 1 |
| Modul | 8 |
| Judul | Array Satu Dimensi dan Sequential Search |
| Bahasa Pemrograman | C |
| Alokasi Waktu | 1 × 170 menit |
| Metode | Demonstrasi, live coding, latihan terbimbing, tracing, eksperimen, debugging, dan latihan mandiri |
| Prasyarat | Modul 1–7 |

## 2. Indikator Keberhasilan

Mahasiswa dinyatakan berhasil apabila mampu:

- mendeklarasikan array dengan kapasitas yang tepat;
- menggunakan indeks dari 0 sampai jumlah_data - 1;
- mengisi array melalui perulangan;
- menampilkan seluruh elemen array;
- mengubah elemen tertentu berdasarkan indeks;
- menghitung total dan rata-rata dengan benar;
- menentukan nilai minimum dan maksimum;
- membuat fungsi yang menerima array;
- membuat fungsi sequential search;
- mengembalikan indeks data yang ditemukan;
- menggunakan -1 sebagai penanda data tidak ditemukan;
- menghitung jumlah kemunculan data;
- menghindari akses pada indeks negatif atau melebihi kapasitas;
- melakukan tracing setiap perbandingan pada proses pencarian;
- menjelaskan kompleksitas sederhana sequential search.

# II. Persiapan Praktikum

## 1. Persiapan Folder Praktikum

Buat struktur folder berikut:

```
Praktikum_Dasar_Pemrograman/
└── Modul_08/
```

```
    ├── latihan_01.c
    ├── latihan_02.c
    ├── latihan_03.c
    ├── latihan_04.c
    ├── tracing.c
    ├── eksperimen.c
    ├── debugging_01.c
    ├── debugging_02.c
    └── tugas_modul_08.c
```

Ketentuan nama berkas:

- menggunakan huruf kecil;
- tidak menggunakan spasi;
- menggunakan garis bawah _;
- menggunakan ekstensi .c.

# III. Konsep Dasar

## 1. Konsep Array Satu Dimensi

Array adalah struktur data yang digunakan untuk menyimpan sejumlah elemen dengan tipe data yang sama menggunakan satu nama variabel.

Tanpa array:

```
int nilai_1;
int nilai_2;
int nilai_3;
int nilai_4;
int nilai_5;
```

Dengan array:

```
int nilai[5];
```

Pernyataan tersebut membuat array bernama nilai yang dapat menyimpan lima bilangan bertipe int.

***Ilustrasi Array***

```
Nama array : nilai

Indeks     :    0      1      2      3      4
              +------+------+------+------+------+
Nilai      : |  80  |  75  |  90  |  65  |  85  |
              +------+------+------+------+------+
```

Elemen pertama:

```
nilai[0]
```

Elemen terakhir:

```
nilai[4]
```

Untuk array berukuran lima, indeks valid adalah: 0, 1, 2, 3, dan 4.

## 2. Deklarasi Array

Bentuk umum:

```
tipe_data nama_array[ukuran];
```

Contoh:

```
int nilai[5];
double harga[10];
char kode[20];
```

Maknanya:

| Deklarasi | Kapasitas |
|---|---|
| int nilai[5]; | 5 elemen int |
| double harga[10]; | 10 elemen double |
| char kode[20]; | 20 elemen char |

Ukuran array harus berupa bilangan bulat positif.

### *Menggunakan Kapasitas Maksimum*

Untuk program praktikum, kapasitas dapat ditetapkan menggunakan macro:

```
#define MAX_DATA 100
```

Kemudian:

```
int data[MAX_DATA];
```

Keuntungannya:

- kapasitas mudah diubah;
- angka batas tidak ditulis berulang;
- maksud program lebih jelas.

Contoh lengkap:

```
#include <stdio.h>

#define MAX_DATA 100

int main(void)
{
    int data[MAX_DATA];

    return 0;
}
```

## 3. Indeks Array

Indeks digunakan untuk mengakses elemen tertentu.

```
nilai[0] = 80;
nilai[1] = 75;
nilai[2] = 90;
```

Menampilkan elemen:

```
printf("%d\n", nilai[0]);
```

Mengubah elemen:

```
nilai[1] = 85;
```

Menyalin elemen:

```
int nilai_pertama = nilai[0];
```

Melakukan perhitungan:

```
int jumlah = nilai[0] + nilai[1];
```

***Batas Indeks***

Apabila array dideklarasikan:

```
int nilai[5];
```

indeks yang valid adalah 0 sampai 4.

Indeks berikut tidak valid:

```
nilai[-1]
nilai[5]
nilai[10]
```

*Mengakses elemen di luar batas array menyebabkan undefined behavior. Program mungkin:*

- menghasilkan nilai yang salah;
- merusak data lain;
- berhenti;
- tampak berjalan normal tetapi sebenarnya tidak aman.

*Compiler tidak selalu dapat mendeteksi kesalahan indeks saat program dijalankan.*

## 4. Inisialisasi Array

***a. Inisialisasi lengkap***

```
int nilai[5] = {80, 75, 90, 65, 85};
```

***b. Ukuran ditentukan otomatis***

```
int nilai[] = {80, 75, 90, 65, 85};
```

Compiler menentukan bahwa array memiliki lima elemen.

***c. Inisialisasi sebagian***

```
int nilai[5] = {80, 75};
```

Hasilnya:

```
nilai[0] = 80
nilai[1] = 75
```

```
nilai[2] = 0
nilai[3] = 0
nilai[4] = 0
```

*Elemen yang tidak diberi nilai secara eksplisit diinisialisasi menjadi nol.*

***d. Menginisialisasi seluruh elemen menjadi nol***

```
int nilai[5] = {0};
```

Semua elemen bernilai nol.

## 5. Mengisi Array Menggunakan Perulangan

```
#include <stdio.h>

#define JUMLAH_DATA 5

int main(void)
{
    int nilai[JUMLAH_DATA];
    int i;

    for (i = 0; i < JUMLAH_DATA; i++)
    {
        printf("Masukkan nilai ke-%d: ", i + 1);
        scanf("%d", &nilai[i]);
    }

    return 0;
}
```

*Perhatikan i = 0 karena indeks pertama array adalah nol.*

*Kondisi i < JUMLAH_DATA membatasi indeks terakhir menjadi JUMLAH_DATA - 1.*

## 6. Menampilkan Array

```
for (i = 0; i < JUMLAH_DATA; i++)
{
    printf("nilai[%d] = %d\n", i, nilai[i]);
}
```

Contoh keluaran:

```
nilai[0] = 80
nilai[1] = 75
nilai[2] = 90
nilai[3] = 65
nilai[4] = 85
```

*Istilah traversal berarti mengunjungi elemen array satu per satu.*

```
for (i = 0; i < jumlah_data; i++)
{
    /* Proses data[i]. */
}
```

## 7. Kapasitas dan Jumlah Data

Perhatikan:

```
#define MAX_DATA 100

int data[MAX_DATA];
int jumlah_data = 5;
```

Kedua nilai tersebut memiliki arti berbeda.

| Istilah | Arti |
|---|---|
| Kapasitas | Banyak elemen maksimal yang dapat disimpan |
| Jumlah data | Banyak elemen yang benar-benar digunakan |

Pada contoh: kapasitas = 100, jumlah data = 5.

*Perulangan pengolahan harus menggunakan jumlah_data, bukan selalu MAX_DATA.*

Benar:

```
for (i = 0; i < jumlah_data; i++)
```

Kurang tepat:

```
for (i = 0; i < MAX_DATA; i++)
```

apabila hanya lima elemen yang telah diisi.

## 8. Menghitung Total dan Rata-Rata

```
int nilai[5] = {80, 75, 90, 65, 85};
int total = 0;
int i;
double rata_rata;

for (i = 0; i < 5; i++)
{
    total += nilai[i];
}

rata_rata = (double) total / 5;
```

Hasil:

```
Total     = 395
Rata-rata = 79.00
```

*Accumulator harus diinisialisasi:*

```
int total = 0;
```

## 9. Menentukan Nilai Minimum dan Maksimum

Gunakan elemen pertama sebagai nilai awal.

```
int minimum = nilai[0];
int maksimum = nilai[0];

for (i = 1; i < jumlah_data; i++)
{
    if (nilai[i] < minimum)
    {
        minimum = nilai[i];
    }

    if (nilai[i] > maksimum)
    {
        maksimum = nilai[i];
    }
}
```

*Perulangan dimulai dari indeks 1 karena elemen indeks 0 sudah digunakan sebagai nilai awal.*

Hindari:

```
int minimum = 0;
```

*karena apabila seluruh data positif dan tidak ada nilai nol, hasil minimum dapat salah.*

## 10. Menghitung Data Berdasarkan Kondisi

Contoh menghitung jumlah mahasiswa lulus:

```
int jumlah_lulus = 0;

for (i = 0; i < jumlah_data; i++)
{
    if (nilai[i] >= 60)
    {
        jumlah_lulus++;
    }
}
```

Menghitung jumlah bilangan genap:

```
int jumlah_genap = 0;

for (i = 0; i < jumlah_data; i++)
{
```

```
    if (data[i] % 2 == 0)
    {
        jumlah_genap++;
    }
}
```

Menghitung frekuensi suatu nilai:

```
int frekuensi = 0;
int target = 75;

for (i = 0; i < jumlah_data; i++)
{
    if (data[i] == target)
    {
        frekuensi++;
    }
}
```

## 11. Array sebagai Parameter Fungsi

Array dapat dikirim ke fungsi.

Prototype:

```
void tampilkan_data(const int data[], int jumlah_data);
```

Definisi:

```
void tampilkan_data(const int data[], int jumlah_data)
{
    int i;

    for (i = 0; i < jumlah_data; i++)
    {
        printf("%d ", data[i]);
    }

    printf("\n");
}
```

Pemanggilan:

```
tampilkan_data(nilai, jumlah_data);
```

Nama array ditulis tanpa indeks ketika dikirim sebagai argumen:

```
nilai
```

Bukan:

```
nilai[i]
```

*karena nilai[i] hanya mengirim satu elemen.*

***Parameter Ukuran***

Fungsi perlu menerima jumlah elemen:

```
void tampilkan_data(const int data[], int jumlah_data);
```

*Array tidak secara otomatis memberi tahu fungsi berapa banyak elemen yang sedang digunakan.*

Oleh karena itu, parameter berikut harus konsisten:

```
data
jumlah_data
```

## 12. Penggunaan const

Apabila fungsi hanya membaca array dan tidak mengubah elemennya, gunakan const.

```
int hitung_total(
    const int data[],
    int jumlah_data
);
```

Definisi:

```
int hitung_total(
    const int data[],
    int jumlah_data
)
{
    int total = 0;
    int i;

    for (i = 0; i < jumlah_data; i++)
    {
        total += data[i];
    }

    return total;
}
```

Penggunaan const:

- menunjukkan bahwa fungsi hanya membaca data;
- mencegah perubahan elemen secara tidak sengaja;
- membuat tujuan fungsi lebih jelas.

Fungsi yang mengisi atau mengubah array tidak menggunakan const:

```
void baca_data(int data[], int jumlah_data);
```

## 13. Perubahan Array di Dalam Fungsi

Berbeda dengan parameter variabel biasa, perubahan elemen array di dalam fungsi dapat terlihat oleh fungsi pemanggil.

```
#include <stdio.h>

void ubah_elemen_pertama(int data[]);

int main(void)
{
    int angka[3] = {10, 20, 30};

    ubah_elemen_pertama(angka);

    printf("%d\n", angka[0]);

    return 0;
}

void ubah_elemen_pertama(int data[])
{
    data[0] = 100;
}
```

Keluaran:

```
100
```

*Secara konseptual, fungsi memperoleh akses ke lokasi elemen array yang sama. Penjelasan lebih mendalam mengenai alamat memori dibahas pada modul pointer.*

## 14. Menyalin Array

Array tidak dapat disalin langsung menggunakan operator penugasan.

Tidak valid:

```
int data_a[5] = {1, 2, 3, 4, 5};
int data_b[5];

data_b = data_a;
```

Gunakan perulangan:

```
for (i = 0; i < 5; i++)
{
    data_b[i] = data_a[i];
}
```

Dapat dibuat fungsi:

```
void salin_array(
    const int sumber[],
    int tujuan[],
    int jumlah_data
)
```

```
{
    int i;

    for (i = 0; i < jumlah_data; i++)
    {
        tujuan[i] = sumber[i];
    }
}
```

## 15. Konsep Sequential Search

Sequential search atau pencarian berurutan adalah algoritma yang memeriksa elemen array satu per satu dari awal sampai:

- data ditemukan; atau
- seluruh elemen selesai diperiksa.

Contoh data:

```
Indeks : 0   1   2   3   4
Data   : 12  25  18  30  21
```

Target: 30

Proses:

1. Bandingkan dengan data[0] = 12 → tidak sama
2. Bandingkan dengan data[1] = 25 → tidak sama
3. Bandingkan dengan data[2] = 18 → tidak sama
4. Bandingkan dengan data[3] = 30 → ditemukan

Data ditemukan pada indeks 3.

## 16. Fungsi Sequential Search

```
int cari_sekuensial(
    const int data[],
    int jumlah_data,
    int target
)
{
    int i;

    for (i = 0; i < jumlah_data; i++)
    {
        if (data[i] == target)
        {
            return i;
        }
    }
```

```
    return -1;
}
```

Pemanggilan:

```
int posisi;

posisi = cari_sekuensial(
    data,
    jumlah_data,
    target
);
```

Pemeriksaan hasil:

```
if (posisi != -1)
{
    printf("Data ditemukan pada indeks %d.\n", posisi);
}
else
{
    printf("Data tidak ditemukan.\n");
}
```

***Mengapa Mengembalikan -1?***

Indeks array yang valid dimulai dari nol.

```
0, 1, 2, ..., jumlah_data - 1
```

*Nilai -1 bukan indeks yang valid sehingga dapat digunakan sebagai penanda: data tidak ditemukan.*

## 17. Pencarian Berhenti Saat Data Pertama Ditemukan

Fungsi berikut menggunakan return i; ketika target ditemukan.

*Artinya, pencarian berhenti pada kemunculan pertama.*

Contoh:

```
Indeks : 0   1   2   3   4
Data   : 10  20  10  30  10
Target : 10
```

Fungsi mengembalikan:

```
0
```

meskipun nilai 10 juga muncul pada indeks 2 dan 4.

## 18. Menghitung Seluruh Kemunculan

Untuk menghitung frekuensi:

```
int hitung_frekuensi(
```

```
    const int data[],
    int jumlah_data,
    int target
)
{
    int frekuensi = 0;
    int i;

    for (i = 0; i < jumlah_data; i++)
    {
        if (data[i] == target)
        {
            frekuensi++;
        }
    }

    return frekuensi;
}
```

Contoh:

```
Data   : 10, 20, 10, 30, 10
Target : 10
```

Hasil:

```
Frekuensi = 3
```

*Fungsi tidak berhenti pada data pertama karena harus memeriksa seluruh elemen.*

## 19. Menampilkan Seluruh Posisi Kemunculan

```
void tampilkan_semua_posisi(
    const int data[],
    int jumlah_data,
    int target
)
{
    int ditemukan = 0;
    int i;

    printf("Target ditemukan pada indeks: ");

    for (i = 0; i < jumlah_data; i++)
    {
        if (data[i] == target)
        {
            printf("%d ", i);
            ditemukan = 1;
        }
```

```
    }

    if (ditemukan == 0)
    {
        printf("Tidak ada");
    }

    printf("\n");
}
```

Contoh:

```
Data   : 10, 20, 10, 30, 10
Target : 10
Posisi : 0 2 4
```

## 20. Efisiensi Sederhana Sequential Search

Sequential search memeriksa data dari awal secara berurutan.

***Kondisi terbaik***

Target ditemukan pada elemen pertama.

```
Jumlah perbandingan = 1
```

***Kondisi terburuk***

Target berada pada elemen terakhir atau tidak ditemukan.

```
Jumlah perbandingan = jumlah data
```

***Kompleksitas intuitif***

Jika jumlah data bertambah dua kali lipat, jumlah pemeriksaan pada kondisi terburuk juga dapat bertambah sekitar dua kali lipat.

Sequential search memiliki kompleksitas waktu:

```
O(n)
```

*Pada tahap ini, notasi tersebut dipahami sebagai pertumbuhan waktu yang sebanding dengan jumlah data.*

Sequential search memiliki kelebihan:

- sederhana;
- mudah diterapkan;
- tidak memerlukan data terurut;
- dapat digunakan pada data kecil.

# IV. Praktikum Terbimbing

## 1. Praktikum Terbimbing 1 — Input dan Output Array

Buat berkas: latihan_01.c. Program:

```
#include <stdio.h>

```

```
#define MAX_DATA 100

void baca_data(int data[], int jumlah_data);
void tampilkan_data(
    const int data[],
    int jumlah_data
);

int main(void)
{
    int data[MAX_DATA];
    int jumlah_data;

    do
    {
        printf("Masukkan jumlah data 1-%d: ", MAX_DATA);
        scanf("%d", &jumlah_data);

        if ((jumlah_data < 1) ||
            (jumlah_data > MAX_DATA))
        {
            printf("Jumlah data tidak valid.\n");
        }
    }
    while ((jumlah_data < 1) ||
           (jumlah_data > MAX_DATA));

    baca_data(data, jumlah_data);

    printf("\nDATA YANG DIMASUKKAN\n");
    tampilkan_data(data, jumlah_data);

    return 0;
}

void baca_data(int data[], int jumlah_data)
{
    int i;

    for (i = 0; i < jumlah_data; i++)
    {
        printf("Data ke-%d: ", i + 1);
        scanf("%d", &data[i]);
    }
}

void tampilkan_data(
    const int data[],
```

```
    int jumlah_data
)
{
    int i;

    for (i = 0; i < jumlah_data; i++)
    {
        printf("data[%d] = %d\n", i, data[i]);
    }
}
```

***Pertanyaan analisis***

1. Apa kapasitas array data?
2. Apa perbedaan kapasitas dan jumlah_data?
3. Mengapa indeks dimulai dari nol?
4. Mengapa tampilan menggunakan i + 1 ketika meminta data?
5. Mengapa fungsi tampilkan_data() menggunakan const?
6. Apakah baca_data() boleh menggunakan const?
7. Apa yang terjadi apabila kondisi perulangan menggunakan i <= jumlah_data?
8. Apa indeks terakhir jika jumlah data adalah 10?

## 2. Praktikum Terbimbing 2 — Statistik Data Array

Buat berkas: latihan_02.c. Program:

```
#include <stdio.h>

#define MAX_DATA 100

void baca_data(double data[], int jumlah_data);

double hitung_total(
    const double data[],
    int jumlah_data
);

double hitung_rata_rata(
    const double data[],
    int jumlah_data
);

double cari_minimum(
    const double data[],
    int jumlah_data
);

double cari_maksimum(
```

```
    const double data[],
    int jumlah_data
);

int main(void)
{
    double data[MAX_DATA];
    int jumlah_data;

    double total;
    double rata_rata;
    double minimum;
    double maksimum;

    do
    {
        printf("Masukkan jumlah data 1-%d: ", MAX_DATA);
        scanf("%d", &jumlah_data);
    }
    while ((jumlah_data < 1) ||
           (jumlah_data > MAX_DATA));

    baca_data(data, jumlah_data);

    total = hitung_total(data, jumlah_data);
    rata_rata = hitung_rata_rata(data, jumlah_data);
    minimum = cari_minimum(data, jumlah_data);
    maksimum = cari_maksimum(data, jumlah_data);

    printf("\nHASIL STATISTIK\n");
    printf("Jumlah data : %d\n", jumlah_data);
    printf("Total       : %.2f\n", total);
    printf("Rata-rata   : %.2f\n", rata_rata);
    printf("Minimum     : %.2f\n", minimum);
    printf("Maksimum    : %.2f\n", maksimum);

    return 0;
}

void baca_data(double data[], int jumlah_data)
{
    int i;

    for (i = 0; i < jumlah_data; i++)
    {
        printf("Data ke-%d: ", i + 1);
        scanf("%lf", &data[i]);
    }
```

```
}

double hitung_total(
    const double data[],
    int jumlah_data
)
{
    double total = 0.0;
    int i;

    for (i = 0; i < jumlah_data; i++)
    {
        total += data[i];
    }

    return total;
}

double hitung_rata_rata(
    const double data[],
    int jumlah_data
)
{
    double total;

    total = hitung_total(data, jumlah_data);

    return total / jumlah_data;
}

double cari_minimum(
    const double data[],
    int jumlah_data
)
{
    double minimum = data[0];
    int i;

    for (i = 1; i < jumlah_data; i++)
    {
        if (data[i] < minimum)
        {
            minimum = data[i];
        }
    }

    return minimum;
}
```

```
double cari_maksimum(
    const double data[],
    int jumlah_data
)
{
    double maksimum = data[0];
    int i;

    for (i = 1; i < jumlah_data; i++)
    {
        if (data[i] > maksimum)
        {
            maksimum = data[i];
        }
    }

    return maksimum;
}
```

***Pertanyaan analisis***

1. Mengapa minimum dan maksimum menggunakan data[0]?
2. Mengapa perulangan minimum dimulai dari indeks 1?
3. Apa yang terjadi apabila jumlah_data bernilai nol?
4. Mengapa jumlah data divalidasi minimal satu?
5. Fungsi mana yang memanggil fungsi lain?
6. Apakah hitung_total() mengubah array?
7. Mengapa semua fungsi statistik menggunakan const?
8. Apakah menghitung total dua kali efisien?
9. Bagaimana menghindari penghitungan total dua kali?

## 3. Praktikum Terbimbing 3 — Sequential Search

Buat berkas: latihan_03.c. Program:

```
#include <stdio.h>

#define MAX_DATA 100

void baca_data(int data[], int jumlah_data);

void tampilkan_data(
    const int data[],
    int jumlah_data
);

int cari_sekuensial(
```

```
    const int data[],
    int jumlah_data,
    int target
);

int hitung_frekuensi(
    const int data[],
    int jumlah_data,
    int target
);

int main(void)
{
    int data[MAX_DATA];
    int jumlah_data;
    int target;
    int posisi;
    int frekuensi;

    do
    {
        printf("Masukkan jumlah data 1-%d: ", MAX_DATA);
        scanf("%d", &jumlah_data);
    }
    while ((jumlah_data < 1) ||
           (jumlah_data > MAX_DATA));

    baca_data(data, jumlah_data);

    printf("\nData: ");
    tampilkan_data(data, jumlah_data);

    printf("Masukkan data yang dicari: ");
    scanf("%d", &target);

    posisi = cari_sekuensial(
        data,
        jumlah_data,
        target
    );

    frekuensi = hitung_frekuensi(
        data,
        jumlah_data,
        target
    );

    if (posisi != -1)
```

```
    {
        printf("Data ditemukan.\n");
        printf("Kemunculan pertama pada indeks: %d\n", posisi);
        printf("Nomor urut data              : %d\n", posisi + 1);
        printf("Jumlah kemunculan            : %d\n", frekuensi);
    }
    else
    {
        printf("Data tidak ditemukan.\n");
    }

    return 0;
}

void baca_data(int data[], int jumlah_data)
{
    int i;

    for (i = 0; i < jumlah_data; i++)
    {
        printf("Data ke-%d: ", i + 1);
        scanf("%d", &data[i]);
    }
}

void tampilkan_data(
    const int data[],
    int jumlah_data
)
{
    int i;

    for (i = 0; i < jumlah_data; i++)
    {
        printf("%d ", data[i]);
    }

    printf("\n");
}

int cari_sekuensial(
    const int data[],
    int jumlah_data,
    int target
)
{
    int i;
```

```
    for (i = 0; i < jumlah_data; i++)
    {
        if (data[i] == target)
        {
            return i;
        }
    }

    return -1;
}

int hitung_frekuensi(
    const int data[],
    int jumlah_data,
    int target
)
{
    int frekuensi = 0;
    int i;

    for (i = 0; i < jumlah_data; i++)
    {
        if (data[i] == target)
        {
            frekuensi++;
        }
    }

    return frekuensi;
}
```

***Pertanyaan analisis***

1. Apa perbedaan posisi indeks dan nomor urut data?
2. Mengapa posisi ditambah satu ketika menampilkan nomor urut?
3. Mengapa cari_sekuensial() berhenti setelah menemukan data pertama?
4. Mengapa hitung_frekuensi() harus memeriksa seluruh array?
5. Apa hasil jika target berada pada indeks 0?
6. Apa hasil jika target berada pada indeks terakhir?
7. Apa hasil jika target tidak ditemukan?
8. Apakah array harus diurutkan sebelum sequential search?
9. Berapa jumlah perbandingan maksimum?

## 4. Praktikum Terbimbing 4 — Menampilkan Seluruh Posisi Data

Buat berkas: latihan_04.c. Program:

```
#include <stdio.h>
```

```
void tampilkan_posisi_target(
    const int data[],
    int jumlah_data,
    int target
);

int main(void)
{
    int data[] = {10, 20, 10, 30, 40, 10, 50};
    int jumlah_data =
        (int) (sizeof(data) / sizeof(data[0]));

    int target;

    printf("Data: ");

    for (int i = 0; i < jumlah_data; i++)
    {
        printf("%d ", data[i]);
    }

    printf("\nMasukkan target: ");
    scanf("%d", &target);

    tampilkan_posisi_target(
        data,
        jumlah_data,
        target
    );

    return 0;
}

void tampilkan_posisi_target(
    const int data[],
    int jumlah_data,
    int target
)
{
    int jumlah_ditemukan = 0;
    int i;

    printf("Target ditemukan pada indeks: ");

    for (i = 0; i < jumlah_data; i++)
    {
        if (data[i] == target)
```

```
        {
            printf("%d ", i);
            jumlah_ditemukan++;
        }
    }

    if (jumlah_ditemukan == 0)
    {
        printf("tidak ada");
    }

    printf("\nJumlah kemunculan: %d\n", jumlah_ditemukan);
}
```

***Menghitung Banyak Elemen Array***

Untuk array yang dideklarasikan dalam fungsi yang sama:

```
int jumlah_data =
    (int) (sizeof(data) / sizeof(data[0]));
```

Maknanya: ukuran seluruh array ÷ ukuran satu elemen.

*Namun, cara tersebut tidak digunakan untuk menentukan ukuran array di dalam fungsi parameter. Fungsi tetap harus menerima jumlah_data secara terpisah.*

***Pertanyaan analisis***

1. Berapa banyak elemen array pada program?
2. Apa hasil pencarian target 10?
3. Apa hasil pencarian target 25?
4. Mengapa program tidak menggunakan return saat target pertama ditemukan?
5. Apa fungsi jumlah_ditemukan?
6. Mengapa sizeof digunakan di dalam main()?
7. Mengapa fungsi tetap menerima jumlah_data?

# V. Eksperimen dan Tracing

## 1. Tracing Sequential Search

Buat berkas: tracing.c. Jangan langsung menjalankan program.

```
#include <stdio.h>

int cari_sekuensial(
    const int data[],
    int jumlah_data,
    int target
);

int main(void)
{
```

```
    int data[] = {12, 7, 25, 18, 30, 9};
    int jumlah_data = 6;
    int target = 18;
    int posisi;

    posisi = cari_sekuensial(
        data,
        jumlah_data,
        target
    );

    printf("Posisi: %d\n", posisi);

    return 0;
}

int cari_sekuensial(
    const int data[],
    int jumlah_data,
    int target
)
{
    int i;

    for (i = 0; i < jumlah_data; i++)
    {
        if (data[i] == target)
        {
            return i;
        }
    }

    return -1;
}
```

Lengkapi trace table berikut.

| Iterasi | i | data[i] | Target | data[i] == target | Tindakan |
|---|---|---|---|---|---|
| 1 | 0 | 12 | 18 | | |
| 2 | 1 | 7 | 18 | | |
| 3 | 2 | 25 | 18 | | |
| 4 | 3 | 18 | 18 | | |

Setelah tracing:

- tentukan indeks yang dikembalikan;
- tentukan jumlah perbandingan;
- prediksi output;

- jalankan program;
- ubah target menjadi 12;
- ulangi tracing;
- ubah target menjadi 9;
- ulangi tracing;
- ubah target menjadi 100;
- ulangi tracing.

## 2. Eksperimen

Buat berkas: eksperimen.c. Gunakan program dasar:

```
#include <stdio.h>

int main(void)
{
    int data[5] = {10, 20, 30, 40, 50};
    int i;

    for (i = 0; i < 5; i++)
    {
        printf("%d ", data[i]);
    }

    printf("\n");

    return 0;
}
```

Lakukan eksperimen berikut satu per satu.

| No. | Perubahan | Prediksi | Hasil Aktual |
|---|---|---|---|
| 1 | Ubah kondisi menjadi i <= 5 | | |
| 2 | Ubah nilai awal menjadi i = 1 | | |
| 3 | Tampilkan data[i + 1] | | |
| 4 | Ubah data[0] menjadi 100 | | |
| 5 | Inisialisasi hanya dua elemen | | |
| 6 | Gunakan int data[5] = {0}; | | |
| 7 | Cari target pada elemen pertama | | |
| 8 | Cari target pada elemen terakhir | | |
| 9 | Cari target yang tidak tersedia | | |
| 10 | Hilangkan return i saat data ditemukan | | |
| 11 | Salin array dengan perulangan | | |
| 12 | Ubah elemen array melalui fungsi | | |

Untuk setiap eksperimen:

- tuliskan prediksi;
- periksa kemungkinan akses di luar batas;
- kompilasi dengan warning aktif;
- jalankan hanya apabila aman;
- catat hasil;
- jelaskan penyebabnya;
- kembalikan program ke bentuk awal.

*Eksperimen pertama dan ketiga dapat mengakses indeks di luar batas. Analisis terlebih dahulu dan tidak perlu dijalankan apabila lingkungan praktikum tidak mendukung pemeriksaan memori.*

# VI. Debugging Challenge

## 1. Debugging Challenge 1 — Pengolahan Array dengan Indeks Salah

Program berikut dimaksudkan untuk menerima lima nilai dan menghitung total serta rata-rata.

```
#include <stdio.h>

#define JUMLAH_DATA 5

int main(void)
{
    int nilai[JUMLAH_DATA];
    int total;
    double rata_rata;
    int i;

    for (i = 1; i <= JUMLAH_DATA; i++)
    {
        printf("Masukkan nilai ke-%d: ", i);
        scanf("%d", &nilai[i]);
    }

    for (i = 0; i <= JUMLAH_DATA; i++)
    {
        total = nilai[i];
    }

    rata_rata = total / JUMLAH_DATA;

    printf("Total     : %d\n", total);
    printf("Rata-rata : %d\n", rata_rata);

    return 0;
}
```

***Kesalahan yang Perlu Ditemukan***

- total belum diinisialisasi;
- input dimulai dari indeks 1;
- kondisi menggunakan <=;
- elemen indeks 0 tidak diisi;
- elemen indeks 5 berada di luar batas;
- accumulator ditimpa, bukan ditambahkan;
- rata-rata menggunakan pembagian bilangan bulat;
- format output rata-rata tidak sesuai.

***Tugas***

- jangan langsung menjalankan program;
- identifikasi indeks yang tidak valid;
- perbaiki perulangan input;
- perbaiki perulangan total;
- inisialisasi accumulator;
- perbaiki perhitungan rata-rata;
- perbaiki format output;
- uji dengan data 60, 70, 80, 90, 100.

Expected output:

```
Total     : 400
Rata-rata : 80.00
```

## 2. Debugging Challenge 2 — Sequential Search yang Salah

Program berikut dimaksudkan untuk mencari target dalam array.

```
#include <stdio.h>

int cari_data(
    int data[],
    int jumlah_data,
    int target
);

int main(void)
{
    int data[6] = {10, 20, 30, 40, 50, 60};
    int target;
    int posisi;

    printf("Masukkan target: ");
    scanf("%d", &target);

    posisi = cari_data(data, 6, target);
```

```
    if (posisi == 0)
    {
        printf("Data tidak ditemukan.\n");
    }
    else
    {
        printf("Data ditemukan pada indeks %d.\n", posisi);
    }

    return 0;
}

int cari_data(
    int data[],
    int jumlah_data,
    int target
)
{
    int i;

    for (i = 0; i <= jumlah_data; i++)
    {
        if (data[i] = target)
        {
            return i;
        }
    }

    return 0;
}
```

***Kesalahan yang Perlu Ditemukan***

- parameter array seharusnya dapat menggunakan const;
- kondisi loop menggunakan <=;
- operator penugasan = digunakan sebagai perbandingan;
- penanda tidak ditemukan menggunakan 0;
- indeks 0 sebenarnya merupakan posisi yang valid;
- pemeriksaan hasil di main() salah;
- fungsi dapat mengakses elemen di luar batas.

***Perbaikan yang Diharapkan***

Gunakan:

```
return -1;
```

untuk data yang tidak ditemukan.

Pemeriksaan:

```
if (posisi == -1)
```

Gunakan:

```
data[i] == target
```

untuk perbandingan.

***Tabel Debugging***

| No. | Bagian Bermasalah | Jenis Kesalahan | Penyebab | Perbaikan |
|---|---|---|---|---|
| 1 | | | | |
| 2 | | | | |
| 3 | | | | |
| 4 | | | | |
| 5 | | | | |
| 6 | | | | |

# VII. Latihan Mandiri

## 1. Level 1 — Membalik Urutan Array

Buat program yang menerima sejumlah data, kemudian menampilkan data tersebut dalam urutan terbalik.

Contoh:

```
Data asli   : 10 20 30 40 50
Data terbalik: 50 40 30 20 10
```

Gunakan perulangan dari indeks terakhir menuju indeks pertama.

## 2. Level 2 — Menghitung Bilangan Genap dan Ganjil dalam Array

Buat program yang menerima sejumlah bilangan bulat, kemudian menghitung jumlah bilangan genap dan ganjil.

Contoh:

```
Data: 4 7 10 13 8

Jumlah genap: 3
Jumlah ganjil: 2
```

## 3. Level 3 — Menghapus Elemen Berdasarkan Nilai

Buat program yang menerima sejumlah data, lalu menampilkan data tersebut setelah satu nilai tertentu dihapus (elemen setelahnya digeser maju).

Contoh:

```
Data awal  : 10 20 30 40 50
Hapus nilai: 30
Data akhir : 10 20 40 50
```

Ketentuan:

- gunakan sequential search untuk menemukan posisi nilai;
- geser elemen setelah posisi tersebut satu langkah ke kiri;

- kurangi jumlah_data sebanyak satu;
- apabila nilai tidak ditemukan, tampilkan pesan tanpa mengubah array.

## 4. Challenge — Menghapus Duplikat

Buat program yang menerima sejumlah data, kemudian menampilkan data tersebut tanpa nilai yang berulang (hanya kemunculan pertama yang dipertahankan).

Contoh:

```
Data awal : 5 3 5 8 3 9 5
Data akhir: 5 3 8 9
```

Petunjuk: untuk setiap elemen, gunakan sequential search pada array hasil untuk memeriksa apakah nilai tersebut sudah pernah dimasukkan.

## 5. Selisih Maksimum Antar Elemen

*Keluaran program pada soal ini akan diperiksa secara ketat: harus sama persis karakter demi karakter dengan Contoh Output.*

Sebuah aplikasi pemantau sensor ingin mengetahui perubahan terbesar antara dua pembacaan berurutan. Buat program C yang menerima sejumlah data, lalu menghitung selisih absolut terbesar antara dua elemen array yang bersebelahan (data[i+1] dan data[i]).

### *Format Input*

Baris pertama: n (jumlah data). Baris kedua: n bilangan bulat dipisah spasi.

### *Batasan*

- $2 \leq n \leq 50$;
- $-1000 \leq$ elemen $\leq 1000$.

### *Contoh 1*

Input:

```
5
10 15 7 20 20
```

Output:

```
========================================
     SELISIH MAKSIMUM ANTAR ELEMEN
========================================
Jumlah data     : 5
Data            : 10 15 7 20 20
----------------------------------------
Selisih maksimum: 13
========================================
```

### *Contoh 2*

Input:

```
4
5 5 5 5
```

Output:

```
=======================================
     SELISIH MAKSIMUM ANTAR ELEMEN
=======================================
Jumlah data     : 4
Data            : 5 5 5 5
---------------------------------------
Selisih maksimum: 0
=======================================
```

***Contoh 3***

Input:

```
2
1 100
```

Output:

```
=======================================
     SELISIH MAKSIMUM ANTAR ELEMEN
=======================================
Jumlah data     : 2
Data            : 1 100
---------------------------------------
Selisih maksimum: 99
=======================================
```

***Petunjuk Pendekatan***

- gunakan array bertipe int dengan kapasitas mencukupi (misalnya #define MAX_DATA 50);
- baca n elemen ke dalam array menggunakan perulangan for;
- gunakan perulangan for (i = 0; i < n - 1; i++) untuk membandingkan data[i+1] dengan data[i], karena elemen terakhir tidak memiliki pasangan setelahnya;
- gunakan fungsi abs() dari <stdlib.h>, atau hitung selisih secara manual dan gunakan if untuk memastikan hasilnya positif;
- cetak seluruh elemen data pada baris “Data” dipisahkan satu spasi, tanpa spasi di akhir baris sebelum baris baru;
- kedua baris pembatas (garis =) harus memiliki jumlah karakter yang sama seperti pada Contoh Output.

*Simpan sebagai latihan_bonus_08.c.*

# VIII. Pengujian Program

## 1. Pengujian Array dan Sequential Search

Program berbasis array dan pencarian harus diuji dari beberapa aspek.

***a. Data pada elemen pertama***

target = data[0]

***b. Data pada elemen terakhir***

target = data[jumlah_data - 1]

***c. Data di tengah array***

target berada pada posisi tengah

***d. Data tidak ditemukan***

target tidak ada dalam array

***e. Data dengan banyak duplikat***

target muncul lebih dari satu kali

***f. Array berisi satu elemen***

jumlah_data = 1

***g. Nilai minimum dan maksimum pada posisi ekstrem***

nilai terkecil di elemen terakhir, nilai terbesar di elemen pertama

***h. Jumlah data pada batas kapasitas***

jumlah_data = MAX_DATA

***i. Input jumlah data tidak valid***

jumlah_data = 0 atau melebihi kapasitas

***Contoh tabel pengujian sequential search***

| No. | Data | Target | Expected Output | Status |
|---|---|---|---|---|
| 1 | 10 20 30 | 10 | Ditemukan indeks 0 | |
| 2 | 10 20 30 | 30 | Ditemukan indeks 2 | |
| 3 | 10 20 30 | 20 | Ditemukan indeks 1 | |
| 4 | 10 20 30 | 99 | Tidak ditemukan | |
| 5 | 10 20 10 20 10 | 10 | Frekuensi 3 | |
| 6 | 7 | 7 | Ditemukan indeks 0 | |

# IX. Tugas Praktikum

## 1. Tugas Praktikum Modul 8 — Program Rekap dan Pencarian Nilai Mahasiswa

Buat program untuk menerima sejumlah nilai mahasiswa menggunakan array, mengolah statistik dasar, dan mencari data tertentu menggunakan sequential search.

***1. Input***

Program menerima jumlah mahasiswa dan nilai setiap mahasiswa.

Batasan:

- jumlah mahasiswa: 1–50;
- nilai setiap mahasiswa: 0–100;
- input yang tidak valid harus diminta kembali.

***2. Proses***

Program menghitung: total nilai, rata-rata nilai, nilai tertinggi beserta posisinya, nilai terendah beserta posisinya, jumlah mahasiswa lulus (nilai ≥ 60).

Program juga menerima satu nilai yang dicari, kemudian:

- menampilkan seluruh posisi (indeks dan nomor urut) mahasiswa dengan nilai tersebut;
- menampilkan jumlah kemunculan nilai tersebut;
- menampilkan pesan yang sesuai apabila nilai tidak ditemukan.

### *3. Contoh Interaksi*

```
================================================
         PROGRAM REKAP NILAI MAHASISWA
================================================
Masukkan jumlah mahasiswa 1-50: 5
Nilai mahasiswa ke-1: 80
Nilai mahasiswa ke-2: 60
Nilai mahasiswa ke-3: 90
Nilai mahasiswa ke-4: 60
Nilai mahasiswa ke-5: 75

HASIL REKAP
------------------------------------------------
Jumlah mahasiswa   : 5
Total nilai        : 365.00
Rata-rata          : 73.00
Nilai tertinggi    : 90.00 (mahasiswa ke-3)
Nilai terendah     : 60.00 (mahasiswa ke-2)
Jumlah lulus       : 5

Masukkan nilai yang dicari: 60

HASIL PENCARIAN
------------------------------------------------
Nilai 60.00 ditemukan pada mahasiswa ke-2, ke-4
Jumlah kemunculan  : 2
================================================
```

### *4. Struktur yang Wajib Digunakan*

Program wajib menggunakan:

- array untuk menyimpan seluruh nilai mahasiswa;
- do-while untuk validasi jumlah mahasiswa;
- for untuk mengisi array;
- do-while untuk validasi setiap nilai;
- fungsi untuk menghitung total dan rata-rata (array sebagai parameter const);
- fungsi untuk mencari nilai tertinggi beserta posisinya;
- fungsi untuk mencari nilai terendah beserta posisinya;
- fungsi sequential search yang menampilkan seluruh posisi kemunculan;
- fungsi untuk menghitung frekuensi kemunculan.

### *5. Tahapan Algoritma*

1. meminta dan memvalidasi jumlah mahasiswa;
2. mengisi array melalui perulangan dengan validasi setiap nilai;
3. menghitung total dan rata-rata melalui fungsi;

4. mencari nilai tertinggi beserta posisinya melalui fungsi;
5. mencari nilai terendah beserta posisinya melalui fungsi;
6. menghitung jumlah mahasiswa lulus;
7. menampilkan hasil rekap;
8. meminta nilai yang dicari;
9. menampilkan seluruh posisi kemunculan melalui fungsi;
10. menampilkan jumlah kemunculan melalui fungsi;
11. menampilkan pesan yang sesuai apabila nilai tidak ditemukan.

### *6. Kerangka Program*

```
#include <stdio.h>

#define MAX_DATA 50

void baca_nilai(double nilai[], int jumlah_data);

double hitung_total(
    const double nilai[],
    int jumlah_data
);

int cari_indeks_tertinggi(
    const double nilai[],
    int jumlah_data
);

int cari_indeks_terendah(
    const double nilai[],
    int jumlah_data
);

int hitung_jumlah_lulus(
    const double nilai[],
    int jumlah_data
);

int hitung_frekuensi(
    const double nilai[],
    int jumlah_data,
    double target
);

/* Lengkapi prototype fungsi yang menampilkan seluruh posisi. */

int main(void)
```

```
{
    double nilai[MAX_DATA];
    int jumlah_data;
    double target;

    /* Validasi jumlah mahasiswa. */

    baca_nilai(nilai, jumlah_data);

    /* Hitung dan tampilkan rekap menggunakan fungsi. */

    printf("\nMasukkan nilai yang dicari: ");
    scanf("%lf", &target);

    /* Tampilkan hasil pencarian menggunakan fungsi. */

    return 0;
}

/* Lengkapi seluruh definisi fungsi. */
```

### *7. Analisis IPO*

Mahasiswa wajib melengkapi tabel berikut.

| **Komponen** | **Uraian** |
|---|---|
| Input | |
| Process | |
| Output | |

### *8. Daftar Fungsi*

Mahasiswa wajib menjelaskan setiap fungsi.

| **Fungsi** | **Parameter** | **Nilai Kembalian** | **Fungsi/Tugas** |
|---|---|---|---|
| baca_nilai() | | | |
| hitung_total() | | | |
| cari_indeks_tertinggi() | | | |
| cari_indeks_terendah() | | | |
| hitung_jumlah_lulus() | | | |
| hitung_frekuensi() | | | |

### *9. Ketentuan Source Code*

Program wajib:

- menggunakan #include <stdio.h>;
- menggunakan int main(void);

- menggunakan array untuk menyimpan nilai;
- menggunakan minimal enam fungsi;
- menggunakan const pada parameter array yang hanya dibaca;
- menggunakan sequential search untuk pencarian;
- menampilkan seluruh posisi kemunculan, bukan hanya yang pertama;
- menghitung frekuensi kemunculan;
- memvalidasi seluruh input;
- menghindari akses di luar batas array;
- menggunakan nama variabel dan fungsi yang deskriptif;
- dikompilasi dengan warning aktif;
- tidak menghasilkan error atau warning penting;
- diuji menggunakan minimal delapan test case;
- disertai trace table untuk satu data uji;
- dapat dijelaskan oleh mahasiswa.

***10. Header Komentar***

```
/*
Nama        :
NIM         :
Kelas       :
Modul       : 8
Deskripsi   : Program rekap dan pencarian nilai mahasiswa
              menggunakan array dan sequential search.
*/
```

***11. Berkas yang Dikumpulkan***

- source code: tugas_modul_08_nim.c
- analisis input–process–output;
- daftar fungsi beserta parameter dan nilai kembalian;
- pseudocode atau flowchart;
- trace table untuk satu data uji;
- tabel pengujian minimal delapan kasus;
- tangkapan layar kompilasi;
- tangkapan layar hasil eksekusi;
- refleksi;
- pernyataan penggunaan AI.

## 2. Contoh Test Case Tugas

| No. | Jumlah Mahasiswa | Data Nilai | Nilai Dicari | Expected Result |
|---|---|---|---|---|
| 1 | 0 | – | – | Input jumlah ditolak |
| 2 | 51 | – | – | Input jumlah ditolak |

| | | | | |
|---|---|---|---|---|
| 3 | 1 | 75 | 75 | Ditemukan pada mahasiswa ke-1 |
| 4 | 3 | 50, 60, 70 | -1 | Input nilai ditolak |
| 5 | 5 | 80, 60, 90, 60, 75 | 60 | Ditemukan 2 kali |
| 6 | 5 | 80, 60, 90, 60, 75 | 100 | Tidak ditemukan |
| 7 | 4 | 100, 100, 100, 100 | 100 | Ditemukan 4 kali |
| 8 | 5 | 80, 60, 90, 60, 75 | 90 | Tertinggi, ditemukan 1 kali |
| 9 | 5 | 80, 60, 90, 60, 75 | 60 | Terendah, ditemukan 2 kali |
| 10 | 3 | 60, 60, 60 | 60 | Tertinggi dan terendah sama |

*Mahasiswa tetap harus menghitung sendiri expected output untuk setiap kasus.*

## 3. Indeks Kemunculan Terakhir

*Buat program C yang menerima sejumlah data dan sebuah target, lalu mencari indeks kemunculan terakhir dari target tersebut di dalam array (bukan kemunculan pertama). Keluaran program dinilai secara ketat: harus sama persis karakter demi karakter, termasuk spasi.*

### *Format Input*

Baris pertama: n (jumlah data). Baris kedua: n bilangan bulat dipisah spasi. Baris ketiga: target yang dicari.

### *Batasan*

- $1 \leq n \leq 50$;
- $-1000 \leq$ elemen dan target $\leq 1000$.

### *Ketentuan*

*Berbeda dengan cari_sekuensial() yang mengembalikan indeks kemunculan pertama menggunakan return i; segera setelah ditemukan, fungsi pada soal ini harus memeriksa seluruh elemen array dan menyimpan indeks setiap kali target ditemukan, sehingga nilai yang tersimpan pada akhir perulangan adalah indeks kemunculan terakhir.*

### *Contoh 1*

Input:

```
5
10 20 10 30 10
10
```

Output:

```
========================================
       INDEKS KEMUNCULAN TERAKHIR
========================================
Jumlah data      : 5
Data             : 10 20 10 30 10
Target           : 10
----------------------------------------
Indeks terakhir  : 4
========================================
```

### *Contoh 2*

Input:

```
3
5 6 7
6
```

Output:

```
========================================
       INDEKS KEMUNCULAN TERAKHIR
========================================
Jumlah data      : 3
Data             : 5 6 7
Target           : 6
----------------------------------------
Indeks terakhir  : 1
========================================
```

***Contoh 3***

Input:

```
3
1 2 3
9
```

Output:

```
========================================
       INDEKS KEMUNCULAN TERAKHIR
========================================
Jumlah data      : 3
Data             : 1 2 3
Target           : 9
----------------------------------------
Indeks terakhir  : -1
========================================
```

***Ketentuan Tambahan***

- Contoh 3 menunjukkan target yang tidak ditemukan — fungsi harus tetap mengembalikan -1, konsisten dengan konvensi cari_sekuensial() pada materi;
- gunakan variabel indeks_terakhir yang diinisialisasi -1 sebelum perulangan, lalu perbarui nilainya setiap kali data[i] == target ditemukan — jangan menggunakan return di dalam perulangan;
- kedua baris pembatas (garis =) harus memiliki jumlah karakter yang sama seperti pada Contoh Output;
- keluaran diperiksa apa adanya (karakter demi karakter), sehingga kesalahan satu spasi tetap dianggap tidak sesuai.

*Simpan sebagai tugas_modul_08_soal2_nim.c dan sertakan sebagai lampiran tambahan bersama tugas utama.*

# X. Kesalahan Umum dan Standar Penulisan

## 1. Kesalahan yang Sering Terjadi

### *a. Indeks dimulai dari 1*

Salah:

```
for (i = 1; i <= jumlah_data; i++)
{
    /* ... */
    data[i]
}
```

Elemen indeks 0 tidak pernah digunakan, dan akses melebihi batas array.

Benar:

```
for (i = 0; i < jumlah_data; i++)
```

***b. Kondisi menggunakan <=***

Salah:

```
for (i = 0; i <= jumlah_data; i++)
```

Mengakses data[jumlah_data] yang berada di luar batas.

Benar:

```
for (i = 0; i < jumlah_data; i++)
```

***c. Accumulator tidak diinisialisasi***

Salah:

```
int total;

for (i = 0; i < jumlah_data; i++)
{
    total += data[i];
}
```

Benar:

```
int total = 0;
```

***d. Menimpa accumulator***

Salah:

```
total = data[i];
```

Benar:

```
total += data[i];
```

***e. Minimum atau maksimum diinisialisasi dengan nol***

Berisiko salah:

```
int minimum = 0;
```

Benar:

```
int minimum = data[0];
```

***f. Menggunakan MAX_DATA, bukan jumlah_data***

Salah:

```
for (i = 0; i < MAX_DATA; i++)
```

apabila array belum terisi penuh.

Benar:

```
for (i = 0; i < jumlah_data; i++)
```

***g. Mengembalikan 0 sebagai penanda tidak ditemukan***

Salah:

```
return 0;
```

Indeks 0 merupakan posisi yang valid.

Benar:

```
return -1;
```

***h. Menggunakan = pada pembandingan pencarian***

Salah:

```
if (data[i] = target)
```

Benar:

```
if (data[i] == target)
```

***i. Berhenti terlalu awal saat menghitung frekuensi***

Salah:

```
for (i = 0; i < jumlah_data; i++)
{
    if (data[i] == target)
    {
        return 1;
    }
}
```

Fungsi frekuensi harus memeriksa seluruh elemen, tidak boleh berhenti pada kemunculan pertama.

***j. Mencoba menyalin array dengan operator =***

Salah:

```
data_b = data_a;
```

Gunakan perulangan untuk menyalin setiap elemen.

***k. Lupa mengirim jumlah_data ke fungsi***

Salah:

```
void tampilkan_data(const int data[]);
```

Fungsi tidak mengetahui berapa banyak elemen yang harus diproses.

Benar:

```
void tampilkan_data(const int data[], int jumlah_data);
```

***l. Mengubah array pada fungsi yang seharusnya hanya membaca***

Tanpa const, kesalahan seperti berikut tidak akan terdeteksi oleh compiler:

```
double hitung_total(double data[], int jumlah_data)
{
    data[0] = 0;

    /* ... */
}
```

Gunakan const agar compiler membantu mencegah perubahan yang tidak disengaja.

***m. Validasi jumlah data diabaikan***

Tanpa validasi, program berisiko menerima jumlah data 0 atau melebihi kapasitas array.

```
if ((jumlah_data < 1) || (jumlah_data > MAX_DATA))
{
    printf("Jumlah data tidak valid.\n");
}
```

***n. Nomor urut dan indeks tertukar***

Indeks dimulai dari 0, sedangkan nomor urut yang ditampilkan kepada pengguna biasanya dimulai dari 1.

```
printf("Data ke-%d\n", indeks + 1);
```

## 2. Standar Penulisan Program Array

***Gunakan konstanta untuk kapasitas***

```
#define MAX_DATA 100
```

Daripada menulis angka 100 secara berulang di banyak tempat.

***Gunakan nama yang membedakan kapasitas dan jumlah data***

```
int data[MAX_DATA];
int jumlah_data;
```

***Gunakan const pada parameter yang hanya dibaca***

```
double hitung_total(const double data[], int jumlah_data);
```

***Gunakan nama indeks yang konsisten***

Untuk perulangan sederhana, i dapat digunakan. Untuk array bersarang atau konteks yang lebih kompleks, nama seperti indeks lebih jelas.

***Pisahkan proses pencarian dan proses tampilan***

Kurang baik:

```
int cari_dan_tampilkan(const int data[], int jumlah_data, int target);
```

Lebih baik:

```
int cari_sekuensial(const int data[], int jumlah_data, int target);
void tampilkan_hasil_pencarian(int posisi);
```

***Gunakan komentar untuk menjelaskan makna -1***

```
// Mengembalikan indeks kemunculan pertama, atau -1 apabila tidak ditemukan.
int cari_sekuensial(const int data[], int jumlah_data, int target);
```

*Komentar tidak perlu menjelaskan setiap baris secara harfiah.*

# XII. Pernyataan Penggunaan AI

**Tidak Menggunakan AI:**

```
Saya menyatakan bahwa tugas ini dikerjakan tanpa menggunakan bantuan
AI generatif.
```

**Menggunakan AI:**

```
Saya menggunakan [nama perangkat AI] untuk membantu menjelaskan
[array, indeks, sequential search, error, atau bagian lain]. Hasil
tersebut saya periksa melalui tracing, kompilasi, pengujian batas
indeks, dan perbandingan dengan algoritma yang disusun.
```

# Daftar Pustaka

Deitel, P., & Deitel, H. (2016). C: How to program (8th ed.). Pearson Education.

Free Software Foundation. (n.d.). GCC, the GNU Compiler Collection: Online documentation. GNU Project. Diakses dari https://gcc.gnu.org/onlinedocs/

International Organization for Standardization. (2018). ISO/IEC 9899:2018 — Information technology — Programming languages — C. ISO.

Kernighan, B. W., & Ritchie, D. M. (1988). The C programming language (2nd ed.). Prentice Hall.

Microsoft. (n.d.). C/C++ for Visual Studio Code. Visual Studio Code Documentation. Diakses dari https://code.visualstudio.com/docs/languages/cpp

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.